



Department of Computer Science and Engineering

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	C01 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Duration	3 Days
Mode	Individual
Student Reg. No.	192425104
Student Name	P. DEEKSHANA

Instructions: Answer all parts. Show the basic operation, every step of each derivation and the final asymptotic result, and interpret what each result means for the platform. Assume the platform's servers execute 10 basic operations per second.

IT APPLICATION-BASED PROBLEM

Scenario – Scalability Audit of “ShopSphere”, an Online Shopping Platform

The following is a complete, assignment-ready answer covering Parts (a) to (g), with formulas, derivations, intermediate steps, asymptotic complexities, numerical calculations, and recommendations.

Given information:

- Current number of products: $n = 100,000$
- Expected number of products: $n = 1,000,000$
- Server processing speed: 10^8 basic operations per second
- Nightly processing limit: 10 minutes = 600 seconds

(a) Understand the Problem

The objective is to analyze the five back-end modules of ShopSphere and determine how their execution time and memory requirements grow with the number of products.

The input size is represented by n , the number of products or records processed. The basic operation is the operation performed most frequently and used to estimate the running time.

Module	Input size	Dominant basic operation	Expected growth	Justification
M1 – Duplicate-Listing Detector	Number of products n	Comparing two product titles and seller IDs	$\Theta(n^2)$	Each product is compared with every later product.
M2 – API Rate Limiter	Number of requests n	Executing <code>Audit(Request[j])</code>	$\Theta(n)$	The inner loop executes $1 + 2 + 4 + \dots$, a geometric series.
M3 – Product Search S1	Number of products n	Comparing the target with a product	$O(n)$	Sequential search may examine every product.
M3 – Product Search S2	Number of indexed products n	Comparing the target with the middle entry	$\Theta(\log n)$	Each comparison reduces the remaining search space by half.
M4 – Recommendation Engine	Number of customer–product records n	Processing and merging partial results	$\Theta(n^2)$ currently; $\Theta(n \log n)$ after redesign	The current recurrence has four subproblems of size $n/2$, while the redesign has two.
M5 – Sales Analytics Engine	Number of sales records n	Sorting and merging revenue records	$\Theta(n \log^2 n)$	The recurrence adds $n \log n$ work at every recursion level.

Interpretation: M1 and the current M4 design have quadratic growth. M2 and S2 scale more efficiently. M5 grows faster than $n \log n$, so it requires careful performance evaluation before the catalogue expands.

(b) Analyse the Iterative Modules

➤ M1 – Duplicate-Listing Detector

The algorithm compares every product with every product that comes after it.

Step 1: Identify the basic operation

The basic operation is the comparison of product titles and seller IDs.

Step 2: Count the comparisons

- For $i = 0$, the inner loop executes $n - 1$ times.
- For $i = 1$, it executes $n - 2$ times.
- For $i = 2$, it executes $n - 3$ times.
- For $i = n - 2$, it executes once.

Therefore, the total number of comparisons is

$$C(n) = (n - 1) + (n - 2) + \cdots + 2 + 1$$

Using the sum of the first $n - 1$ natural numbers,

$$C(n) = \frac{(n - 1)n}{2}$$

Hence, the exact number of comparisons is

$$C(n) = \frac{n(n - 1)}{2}$$

Step 3: Determine time complexity

Expanding the expression,

$$C(n) = \frac{n^2 - n}{2}$$

The dominant term is $n^2/2$. Therefore,

$$T(n) = \Theta(n^2)$$

The algorithm requires quadratic time because it compares pairs of products.

Step 4: Determine space complexity

The algorithm uses loop variables and does not create an additional data structure proportional to n .

$$S(n) = \Theta(1)$$

This is the auxiliary space complexity, excluding the storage of the product catalogue and any separate duplicate-marker storage.

Interpretation: The number of comparisons grows approximately fourfold whenever the input size doubles. This makes M1 a major scalability concern.

➤ **M2 – API Rate Limiter**

The algorithm begins with $i = 1$ and doubles i after each iteration. The inner loop performs i audit operations.

Step 1: Identify the values of i

The values are

$$1, 2, 4, 8, 16, \dots$$

The loop continues while $i < n$.

Let k be the number of outer-loop iterations. The value of i in iteration k , starting from iteration 1, is 2^{k-1} .

The loop terminates when

$$2^k \geq n$$

Therefore,

$$k = \lceil \log_2 n \rceil$$

Step 2: Derive the exact number of audit operations

The total number of operations is

$$A(n) = 1 + 2 + 4 + \dots + 2^{k-1}$$

Using the geometric-series formula,

$$A(n) = \frac{2^k - 1}{2 - 1}$$

Substituting $k = \lceil \log_2 n \rceil$,

$$\boxed{A(n) = 2^{\lceil \log_2 n \rceil} - 1}$$

For n equal to a power of two, this simplifies to $A(n) = n - 1$.

Step 3: Check the team member's claim

The team member claims that the complexity is $\Theta(n \log n)$, because the outer loop executes approximately $\log n$ times and the inner loop executes up to n times.

This reasoning incorrectly assumes that every inner-loop iteration processes n requests. In reality, the inner loop sizes are 1,2,4,8,

Since

$$n \leq 2^{\lceil \log_2 n \rceil} < 2n$$

we get

$$n - 1 \leq A(n) < 2n - 1$$

Thus, the tight asymptotic bound is

$$\boxed{T(n) = \Theta(n)}$$

The claim is incorrect.

Step 4: Verify for $n = 1,024$

Since

$$1024 = 2^{10}$$

the values of i are 1,2,4, ...,512.

The number of audit operations is

$$A(1024) = 1 + 2 + 4 + \dots + 512$$

$$A(1024) = 2^{10} - 1$$

$$\boxed{A(1024) = 1,023 \text{ operations}}$$

Interpretation: M2 scales linearly and is suitable for processing a large number of requests, assuming each audit operation takes a constant amount of time.

(c) Solve a Recurrence by Substitution

➤ **M3 – Product Search S2**

The sorted product index uses binary search. Each comparison discards half of the remaining search space.

Given:

$$T(n) = T(n/2) + 1$$

with

$$T(1) = 1$$

We solve the recurrence using backward substitution, assuming n is a power of two.

Step 1: Expand the recurrence

First expansion:

$$T(n) = T(n/2) + 1$$

Second expansion:

$$T(n) = T(n/4) + 1 + 1$$

$$T(n) = T(n/4) + 2$$

Third expansion:

$$T(n) = T(n/8) + 1 + 2$$

$$T(n) = T(n/8) + 3$$

After k expansions,

$$\boxed{T(n) = T(n/2^k) + k}$$

Step 2: Find the stopping condition

The recurrence reaches the base case when the input size becomes 1.

$$\frac{n}{2^k} = 1$$

Therefore,

$$n = 2^k$$

Taking logarithms to base 2,

$$k = \log_2 n$$

Step 3: Substitute into the recurrence

We have

$$T(n) = T(1) + \log_2 n$$

Since $T(1) = 1$,

$$T(n) = \log_2 n + 1$$

This is the exact closed form for powers of two under the given recurrence and base condition.

Step 4: Determine time complexity

The dominant term is $\log_2 n$.

$$T(n) = \Theta(\log n)$$

Binary search is efficient because each comparison eliminates half the remaining candidates.

Step 5: Determine space complexity

The space complexity depends on whether the search is implemented iteratively or recursively.

Implementation	Time complexity	Auxiliary space complexity
Iterative binary search	$\Theta(\log n)$	$\Theta(1)$
Recursive binary search	$\Theta(\log n)$	$\Theta(\log n)$

Explanation: Iterative binary search uses a fixed number of variables. Recursive binary search creates a function-call stack, which grows with the number of recursive calls.

For $n = 1,000,000$,

$$\log_2(1,000,000) \approx 19.93$$

Thus, binary search requires approximately 20 comparisons in the worst case, depending on the precise implementation.

(d) Apply the Master Theorem

The Master Theorem helps determine the time complexity of divide-and-conquer algorithms of the form

$$T(n) = aT(n/b) + f(n)$$

where:

- a is the number of recursive subproblems.
- b is the factor by which the input size is reduced.
- $f(n)$ is the non-recursive work.
- $n^{\log_b a}$ is the comparison term used to select the theorem's case.

➤ **M4 – Current Recommendation Engine**

Given:

$$T(n) = 4T(n/2) + n$$

Step 1: Identify the parameters

$$a = 4, b = 2, f(n) = n$$

Calculate the critical term:

$$n^{\log_b a} = n^{\log_2 4}$$

Since $\log_2 4 = 2$,

$$n^{\log_2 4} = n^2$$

Step 2: Compare $f(n)$ with the critical term

We have

$$f(n) = n$$

and

$$n < n^2$$

Therefore,

$$f(n) = O(n^{2-1})$$

This satisfies Master Theorem Case 1, where the non-recursive work is polynomially smaller than the critical term.

Hence,

$$T(n) = \Theta(n^2)$$

Interpretation: The current recommendation engine has quadratic running time because of its four recursive subproblems.

➤ **M4 – Proposed Recommendation Engine**

The redesigned algorithm removes replication and uses

$$T(n) = 2T(n/2) + n$$

Step 1: Identify the parameters

$$a = 2, b = 2, f(n) = n$$

Calculate the critical term:

$$n^{\log_b a} = n^{\log_2 2} = n$$

Therefore,

$$f(n) = \Theta(n^{\log_b a})$$

This satisfies Master Theorem Case 2.

Hence,

$$T(n) = \Theta(n \log n)$$

Interpretation: Removing replication reduces the complexity from quadratic to $n \log n$, substantially improving scalability.

Effect of doubling the input size

Design	Complexity	Approximate effect when n doubles
Current	$\Theta(n^2)$	Running time becomes 4 times larger.
Proposed	$\Theta(n \log n)$	Running time becomes slightly more than 2 times larger.

For the proposed design, the asymptotic growth ratio is

$$\frac{2n \log_2(2n)}{n \log_2 n}$$

Since $\log_2(2n) = \log_2 n + 1$,

$$= 2 \left(\frac{\log_2 n + 1}{\log_2 n} \right)$$

$$\boxed{= 2 \left(1 + \frac{1}{\log_2 n} \right)}$$

As n increases, the ratio approaches 2.

➤ **M5 – Sales Analytics Engine**

Given:

$$T(n) = 2T(n/2) + n \log n$$

Step 1: Identify the parameters

$$a = 2, b = 2, f(n) = n \log n$$

The critical term is

$$n^{\log_b a} = n^{\log_2 2} = n$$

Step 2: Compare $f(n)$ with the critical term

$$f(n) = n \log n$$

This is not polynomially smaller than n , nor is it simply $\Theta(n)$. Therefore, none of the three basic Master Theorem cases, in their standard forms, directly gives the answer.

Step 3: Apply the extended Master Theorem

The extended form states that when

$$f(n) = \Theta(n^{\log_b a} \log^k n)$$

for $k \geq 0$, the solution is

$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$$

For M5,

$$n^{\log_b a} = n$$

and

$$f(n) = n \log n = n \log^1 n$$

Therefore, $k = 1$.

Substituting,

$$T(n) = \Theta(n \log^{1+1} n)$$

Hence,

$$T(n) = \Theta(n \log^2 n)$$

Interpretation: M5 grows faster than $n \log n$ but much more slowly than n^2 . Its performance should be monitored as the sales catalogue grows.

(e) Analyse Best, Average and Worst Cases

Best case is the minimum number of comparisons, average case is the expected number of comparisons, and worst case is the maximum number of comparisons.

➤ M3 – Search Version S1: Sequential Search

Sequential search checks each product from the beginning until the target is found or the catalogue ends.

Case	Input condition	Number of comparisons
Best	Target is the first product	1
Average, successful search	Target is equally likely to be at any position	$\frac{n+1}{2}$
Worst	Target is the last product or does not exist	n

Therefore, the worst-case time complexity is

$$T(n) = \Theta(n)$$

❖ Derive the average number of comparisons when probability is p

Let:

- p = probability that the product exists.

- $1 - p$ = probability that the product does not exist.
- $\frac{n+1}{2}$ = average comparisons when the product exists.
- n = comparisons when the product does not exist.

Using the expected-value formula,

$$E[C] = p\left(\frac{n+1}{2}\right) + (1-p)n$$

Expanding,

$$E[C] = \frac{p(n+1)}{2} + n - pn$$

Simplifying,

$$E[C] = n - \frac{p(n-1)}{2}$$

Evaluate for $n = 1,000,000$ and $p = 0.5$

Substitute the values:

$$E[C] = 1,000,000 - \frac{0.5(1,000,000 - 1)}{2}$$

$$= 1,000,000 - \frac{499,999.5}{2}$$

$$= 1,000,000 - 249,999.75$$

Therefore,

$$E[C] = 750,000.25$$

The expected number of comparisons is approximately 750,000 per search.

➤ **M3 – Search Version S2: Binary Search**

Binary search repeatedly compares the target with the middle entry of a sorted index and discards half the remaining entries.

Case	Input condition	Number of comparisons
Best	Target is at the middle entry	1
Average	Target is found after a typical sequence of halvings	Approximately $\log_2 n$
Worst	Target is absent or found at the deepest search level	At most $\lceil \log_2(n+1) \rceil$ for a standard binary-search implementation

The worst-case complexity is

$$T(n) = \Theta(\log n)$$

For $n = 1,000,000$,

$$\log_2 n \approx 19.93$$

Thus, binary search needs approximately 20 comparisons in the worst case.

Comparison: Sequential search may require nearly one million comparisons, while binary search requires about 20. This makes S2 the preferred choice when the product index is sorted and maintained correctly.

(f) Prove an Asymptotic Bound

The nightly report generator performs

$$f(n) = 3n^2 + 10n\log_2 n + 50$$

We must prove that $f(n) = \Theta(n^2)$ using the formal definitions and show that $f(n) \notin O(n)$.

Step 1: Recall the definition of Big-Theta

A function $f(n)$ is $\Theta(g(n))$ if there exist positive constants c_1, c_2 and an integer n_0 such that

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$$

for every $n \geq n_0$.

Here, choose

$$g(n) = n^2$$

We need to find suitable constants c_1, c_2, n_0 .

Step 2: Establish the lower bound

Since all terms are non-negative for $n \geq 1$,

$$f(n) = 3n^2 + 10n\log_2 n + 50$$

Therefore,

$$f(n) \geq 3n^2$$

So we can choose

$$\boxed{c_1 = 3}$$

Step 3: Establish the upper bound

For $n \geq 1$,

$$\log_2 n \leq n$$

Also,

$$1 \leq n^2$$

Therefore,

$$n\log_2 n \leq n^2$$

and

$$50 \leq 50n^2$$

Using these inequalities,

$$f(n) = 3n^2 + 10n\log_2 n + 50$$

$$f(n) \leq 3n^2 + 10n^2 + 50n^2$$

$$f(n) \leq 63n^2$$

Thus, we can choose

$$\boxed{c_2 = 63}$$

Step 4: Select n_0

Both inequalities hold for all $n \geq 1$. Therefore,

$$\boxed{n_0 = 1}$$

The required constants are:

$$c_1 = 3, c_2 = 63, n_0 = 1$$

Combining the bounds,

$$3n^2 \leq f(n) \leq 63n^2$$

for every $n \geq 1$.

Hence, by the formal definition,

$$f(n) = \Theta(n^2)$$

Step 5: Prove that $f(n)$ is not $O(n)$

By the definition of Big-O, if $f(n) = O(n)$, there must exist constants $c > 0$ and n_0 such that

$$f(n) \leq cn$$

for every $n \geq n_0$.

But,

$$f(n) = 3n^2 + 10n\log_2 n + 50$$

Since $f(n) \geq 3n^2$, the assumption would imply

$$3n^2 \leq cn$$

Dividing both sides by $n > 0$,

$$3n \leq c$$

or

$$n \leq \frac{c}{3}$$

This cannot hold for all sufficiently large n , because n can grow without limit while c is a fixed constant.

Therefore, the assumption is false.

$$f(n) \notin O(n)$$

Conclusion: The report generator has quadratic asymptotic growth, not linear growth. Its dominant term is $3n^2$.

(g) Scalability Decision

The expected catalogue size is $n = 1,000,000$. Each server executes 10^8 basic operations per second, and the nightly processing window is 600 seconds.

The estimated running time is calculated using

$$\text{Running time} = \frac{\text{Number of basic operations}}{10^8}$$

Step 1: M1 – Duplicate-Listing Detector

The exact number of comparisons is

$$C(n) = \frac{n(n-1)}{2}$$

Substituting $n = 1,000,000$,

$$C(n) = \frac{1,000,000(999,999)}{2}$$

$$\boxed{C(n) = 499,999,500,000}$$

Running time:

$$t = \frac{499,999,500,000}{10^8}$$

$$\boxed{t = 4,999.995 \text{ seconds}}$$

Converting to minutes,

$$t = \frac{4,999.995}{60}$$

$$\boxed{t \approx 83.33 \text{ minutes}}$$

Decision: Fails the 10-minute limit. M1 must be redesigned.

Recommendation: Use a hash table keyed by a normalized combination of product title and seller ID. Products with the same key can be grouped together, avoiding comparisons between every possible pair. Expected processing time can be reduced to $\Theta(n)$, assuming efficient hashing and suitable handling of duplicate records.

Step 2: M2 – API Rate Limiter

The exact operation count is

$$A(n) = 2^{\lceil \log_2 n \rceil} - 1$$

Since

$$\lceil \log_2 1,000,000 \rceil = 20$$

we get

$$A(n) = 2^{20} - 1$$

$$\boxed{A(n) = 1,048,575}$$

Running time:

$$t = \frac{1,048,575}{10^8}$$

$$\boxed{t \approx 0.01049 \text{ seconds}}$$

Decision: Meets the 10-minute limit. No complexity-based redesign is required.

Step 3: M3 – Product Search S2

Binary search takes approximately 20 comparisons in the worst case.

$$C(n) \approx \lceil \log_2(1,000,000) \rceil$$

$$\boxed{C(n) \approx 20}$$

Running time:

$$t \approx \frac{20}{10^8}$$

$$\boxed{t \approx 0.0000002 \text{ seconds}}$$

This is the idealized time for 20 basic comparisons. Actual latency will be higher because of memory

access, network requests, and other system overheads.

Decision: Easily meets the processing limit. Retain the sorted index and binary-search implementation.

Step 4: M4 – Current Recommendation Engine

The current recurrence is

$$T(n) = 4T(n/2) + n$$

Its complexity is

$$T(n) = \Theta(n^2)$$

Using n^2 as an order-of-growth estimate,

$$\text{Operations} \approx (1,000,000)^2$$

$$\boxed{\text{Operations} \approx 10^{12}}$$

Estimated time:

$$t \approx \frac{10^{12}}{10^8}$$

$$\boxed{t \approx 10,000 \text{ seconds}}$$

Converting to minutes,

$$t \approx 166.67 \text{ minutes}$$

This is an order-of-growth estimate; the exact count depends on the base case and implementation constants.

Decision: The current design is expected to fail the 10-minute limit.

Recommendation: Remove unnecessary replicated subproblems and adopt the proposed recurrence $T(n) = 2T(n/2) + n$. This reduces the complexity from $\Theta(n^2)$ to $\Theta(n \log n)$.

Step 5: M4 – Proposed Recommendation Engine

The redesigned recurrence is

$$T(n) = 2T(n/2) + n$$

Its complexity is

$$T(n) = \Theta(n \log_2 n)$$

For $n = 1,000,000$,

$$n \log_2 n = 1,000,000 \log_2(1,000,000)$$

$$\boxed{\text{Operations} \approx 19,931,569}$$

Estimated time:

$$t \approx \frac{19,931,569}{10^8}$$

$$\boxed{t \approx 0.1993 \text{ seconds}}$$

This estimate uses the $n \log n$ work term; the complete recurrence may include additional leaf-processing work.

Decision: The redesigned version comfortably meets the 10-minute limit under the stated operation-rate assumption.

Step 6: M5 – Sales Analytics Engine

The recurrence is

$$T(n) = 2T(n/2) + n \log_2 n$$

The complexity is

$$T(n) = \Theta(n \log^2 n)$$

Using the leading growth term for the estimate,

$$\text{Operations} \approx n(\log_2 n)^2$$

Substituting $n = 1,000,000$,

$$\text{Operations} \approx 1,000,000(19.93157)^2$$

$$\boxed{\text{Operations} \approx 397,266,000}$$

Estimated time:

$$t \approx \frac{397,266,000}{10^8}$$

$$\boxed{t \approx 3.97 \text{ seconds}}$$

This is an order-of-growth estimate, not an exact count of every comparison and data movement.

Decision: M5 is expected to meet the 10-minute limit. It should be monitored, and its implementation should avoid unnecessary repeated sorting.

Step 7: Comparison of all modules

Estimated workload for one million products or records, using the given processing rate.

Module/design	Estimated operations	Estimated time	10-minute limit
M1 – Duplicate detector	4.999995×10^{11}	83.33 min	Fails
M2 – Rate limiter	1,048,575	0.0105 sec	Meets
M3 – S2 binary search	About 20 comparisons/search	0.0000002 sec/search	Meets
M4 – Current	Order of 10^{12}	About 166.67 min	Fails
M4 – Proposed	About 1.993×10^7	0.199 sec	Meets
M5 – Sales analytics	Order of 3.973×10^8	About 3.97 sec	Meets

The M3 estimate is for one search, whereas the other figures describe the corresponding routine's estimated workload. These figures assume the stated basic-operation rate and omit real-world overheads. The M1 and M4 current-design times are approximate asymptotic estimates rather than exact runtime predictions.

❖ Final Scalability Recommendations

Based on the analysis, the following actions should be taken before ShopSphere expands to one million products.

1. Redesign M1 – Duplicate-Listing Detector. Replace the nested-loop comparison with hash-based

grouping using product title and seller ID. This can reduce expected time complexity from $\Theta(n^2)$ to $\Theta(n)$.

2. Redesign M4 – Recommendation Engine. Remove unnecessary replication and adopt the proposed divide-and-conquer approach. The complexity improves from $\Theta(n^2)$ to $\Theta(n \log n)$.

3. Retain M2 – API Rate Limiter. Its tight bound is $\Theta(n)$, and the expected operation count is well within the processing limit.

4. Retain M3 S2 – Binary Search. It provides $\Theta(\log n)$ search time, provided the product index remains sorted and updated.

5. Maintain and monitor M5 – Sales Analytics Engine. Its $\Theta(n \log^2 n)$ complexity is expected to fit the nightly window under the given processing assumptions. Avoid redundant sorting and profile actual execution times as data volume grows.

❖ Overall Conclusion

The scalability audit shows that algorithm selection has a major impact on ShopSphere's performance. The duplicate-listing detector and the current recommendation engine are the two modules that are expected to exceed the 10-minute nightly processing window at one million products.

Redesigning M1 with hash-based grouping and M4 with reduced replication will significantly improve processing efficiency. M2, M3 S2, and M5 can be retained, with performance monitoring where appropriate.

These changes will help ShopSphere accommodate catalogue growth while maintaining efficient product search, recommendation generation, duplicate detection, and sales analytics.

Rubrics – Application-Based Question

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements.	Good understanding with minor gaps.	Basic understanding; some requirements unclear.	Poor understanding or misinterpretation of the problem.
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems.	Applies relevant concepts correctly with minor errors.	Applies basic concepts with limited accuracy.	Fails to apply appropriate concepts.
Problem-Solving Approach	20	Logical, well-structured, and efficient approach.	Mostly logical approach with minor gaps.	Basic approach with some inconsistencies.	Illogical or unclear approach.
Accuracy of Solution	20	Completely correct solution with proper justification.	Mostly correct with minor errors.	Partially correct solution.	Incorrect or incomplete solution.
Clarity	10	Clear, well-organized, and properly explained solution.	Understandable with minor clarity issues.	Limited clarity and explanation.	Poorly presented and difficult to understand

Code of Conduct

I **P. DEEKSHANA (192425104)** (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: **P. Deekshana** Date: **9.10.2026**